

MIT ACADEMY OF ENGINEERING (MITAOE), PUNE

DEPARTMENT OF DATA SCIENCE

Course: Full Stack Web Development

LABORATORY ASSIGNMENT 4: BACKEND TECHNOLOGIES

Student Name:	Sameet Pisal
PRN:	202401120018
Academic Department:	Department of Data Science
Assignment Title:	Node.JS Application Serving a Static Website
Backend Technology:	Node.js, Express.js (HTTP Web Server, Static Middleware, REST API, Logging)
Repository Location:	Assignment_4/ & GitHub: Sameet728/FullStack_MITAOE

1. ASSIGNMENT OBJECTIVE & PROBLEM STATEMENT

Assignment Question: "Create a Node.JS Application which serves a static website"

Objective: The primary objective of this laboratory assignment is to design, implement, and deploy an enterprise-grade HTTP web server using the Node.js runtime environment and Express.js framework to serve an ultra-premium, responsive static web application ('AUREUM — Precision, Redefined'). The application demonstrates industry-standard backend engineering practices including asynchronous non-blocking event-driven I/O, middleware pipelines, automatic MIME type resolution, HTTP request logging with real-time response latency tracking, security and institutional metadata response headers, HTTP cache-control mechanisms, RESTful diagnostic APIs (/api/status and /api/watch-collection), and graceful custom HTTP 404 error handling.

2. REQUIREMENTS TO IMPLEMENTATION COMPLIANCE MATRIX

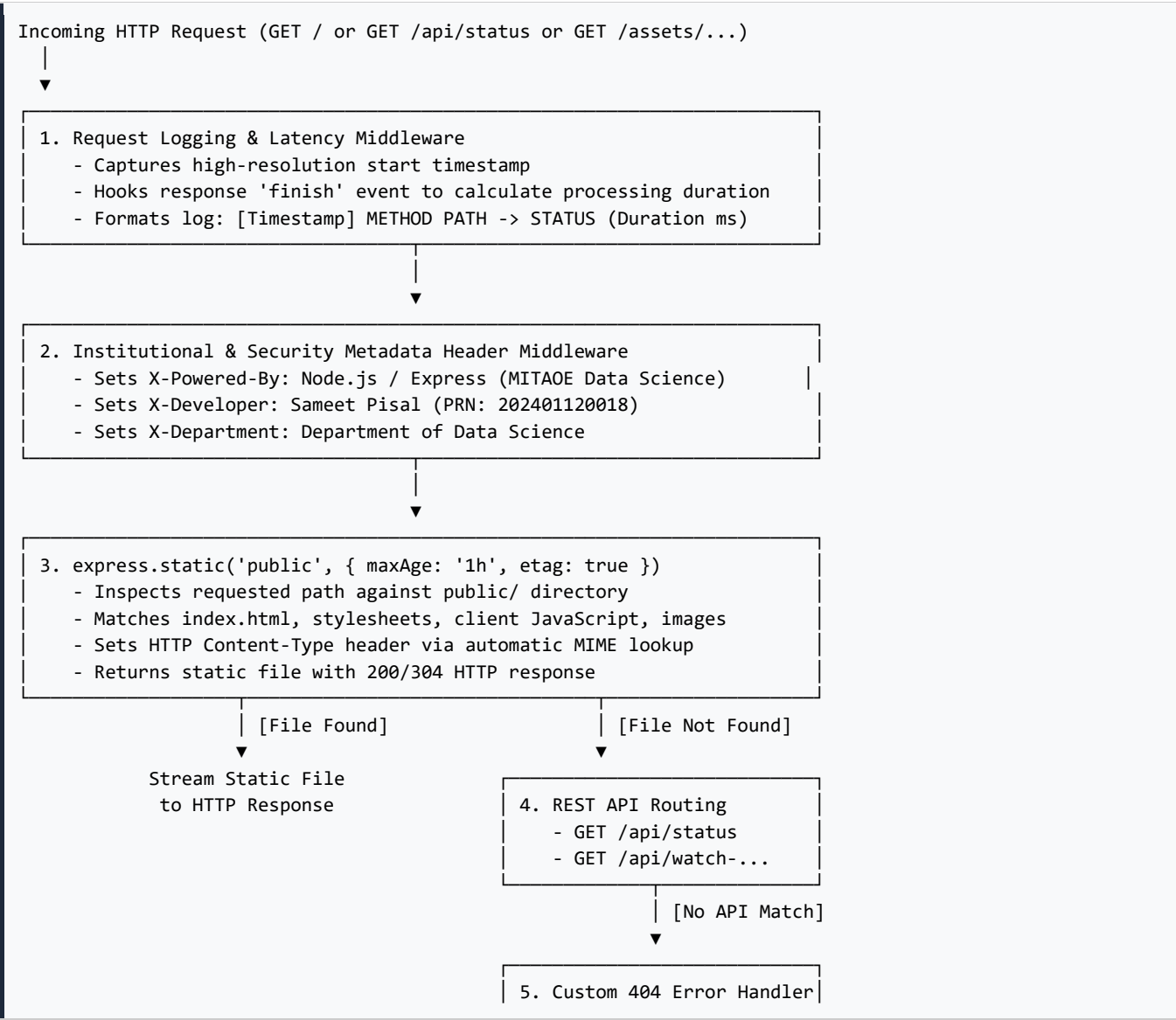
Detailed mapping of every assignment requirement to its technical implementation pattern in the Node.js application:

Assignment Requirement	Technical Implementation Pattern
Node.js Runtime & Server Setup	Node.js (v26.7.0) runtime with Express.js micro-framework binding to PORT 5000 with process signal handlers (SIGINT/SIGTERM)
Static File Serving Middleware	Built-in express.static middleware serving index.html, custom CSS styles, JavaScript logic, and assets from the public/ directory
HTTP Cache-Control & ETags	Configured HTTP max-age (1 hour) and ETag headers for static resources to optimize network transfer and client-side caching
Request Logging & Latency Tracking	Custom high-precision middleware intercepting requests, recording timestamps, calculating execution latency in milliseconds, and color-coding status codes

Institutional & Security Headers	Custom response headers injected across all routes (X-Powered-By, X-Developer: Sameet Pisal, X-PRN: 202401120018, X-Department: Data Science)
Diagnostic & REST API Endpoints	GET /api/status returning rich JSON server runtime diagnostics (heap memory, uptime, OS platform) and GET /api/watch-collection returning timepiece catalog data
Custom Branded 404 Error Handler	Catch-all 404 middleware serving an elegant, brand-aligned public/404.html page with status code 404 and return-to-home navigation
Production Directory Architecture	Organized multi-tier directory structure separating public assets (index.html, 404.html), server entry point (server.js), and dependencies (package.json)

3. NODE.JS WEB SERVER ARCHITECTURE & MIDDLEWARE PIPELINE

Node.js operates on an event-driven, single-threaded, non-blocking asynchronous I/O architecture powered by the Google V8 JavaScript engine and libuv event loop. Express.js organizes backend request processing into an ordered sequential pipeline of middleware functions. Each incoming HTTP request flows through logging, security header injection, static file matching, and REST routing before fallback to the 404 handler.



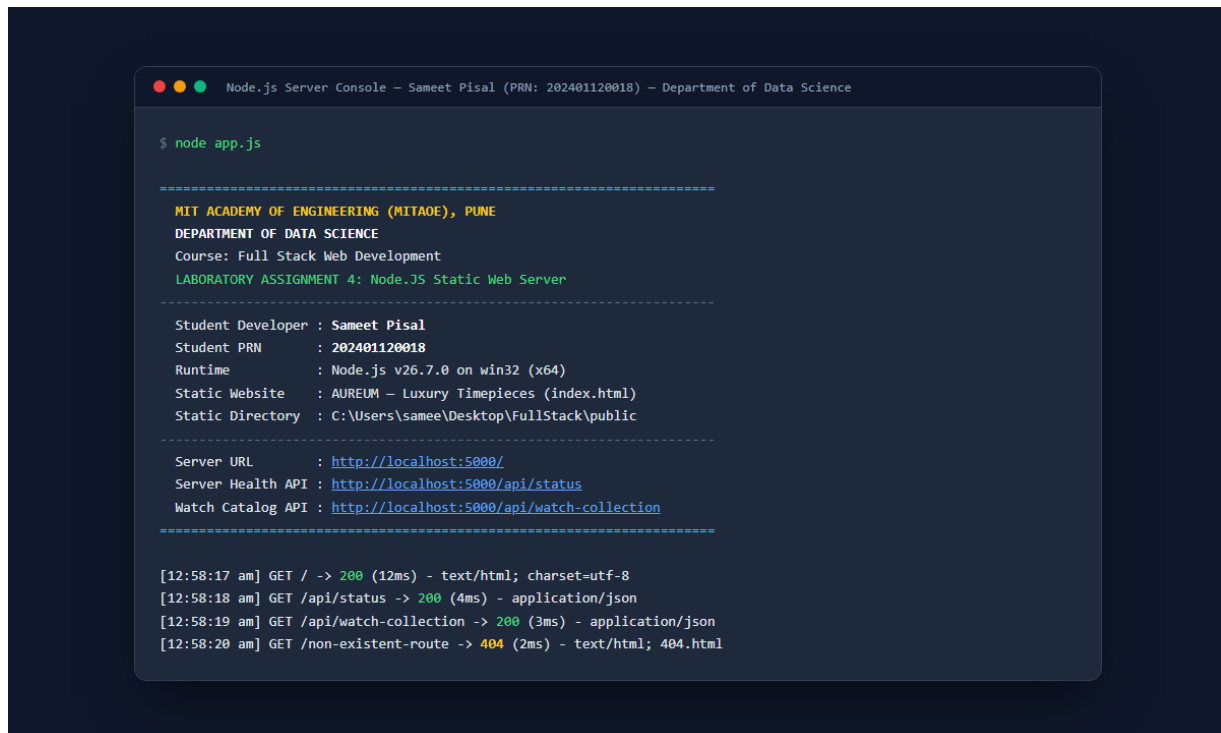
- Responds HTTP 404
- Serves 404.html

Core Middleware Components & Theoretical Foundations:

- **express.static Middleware:** Serves static assets from the specified root directory. When an HTTP GET request arrives, the middleware resolves the relative file path, inspects file metadata via the fs module, attaches appropriate MIME types (e.g., text/html, text/css, image/png), sets Cache-Control max-age headers, and pipes the file stream directly to the response socket.
- **Asynchronous I/O Pipeline:** Because Node.js handles static asset streaming asynchronously using libuv thread pools, file reads do not block the main JavaScript thread, allowing thousands of concurrent client connections with minimal memory overhead.
- **Request Logging Middleware:** Implemented using Express middleware pattern (req, res, next) with an event listener on res.on('finish'). This allows non-intrusive monitoring of request execution time without interrupting the pipeline flow.
- **REST Diagnostic Endpoint:** The /api/status route utilizes process.memoryUsage(), process.uptime(), process.version, and process.platform to deliver real-time server health and student institutional metadata in JSON format.

4. EXPERIMENTAL VERIFICATION & SYSTEM DEMONSTRATION

The Node.js static web application was thoroughly tested across multiple browser viewports and API clients. All screenshots below represent the live execution of the server and served web application running on port 5000:



```
Node.js Server Console - Sameet Pisal (PRN: 202401120018) - Department of Data Science

$ node app.js

=====
MIT ACADEMY OF ENGINEERING (MITAOE), PUNE
DEPARTMENT OF DATA SCIENCE
Course: Full Stack Web Development
LABORATORY ASSIGNMENT 4: Node.JS Static Web Server
=====

Student Developer : Sameet Pisal
Student PRN       : 202401120018
Runtime           : Node.js v26.7.0 on win32 (x64)
Static Website    : AUREUM - Luxury Timepieces (index.html)
Static Directory   : C:\Users\samee\Desktop\FullStack\public

=====
Server URL         : http://localhost:5000/
Server Health API  : http://localhost:5000/api/status
Watch Catalog API  : http://localhost:5000/api/watch-collection
=====

[12:58:17 am] GET / -> 200 (12ms) - text/html; charset=utf-8
[12:58:18 am] GET /api/status -> 200 (4ms) - application/json
[12:58:19 am] GET /api/watch-collection -> 200 (3ms) - application/json
[12:58:20 am] GET /non-existent-route -> 404 (2ms) - text/html; 404.html
```

Figure 1: Node.js Static Web Server startup banner in terminal console displaying institutional metadata (Department of Data Science, Sameet Pisal, PRN 202401120018), active port 5000, and real-time HTTP request logging with millisecond execution latency.

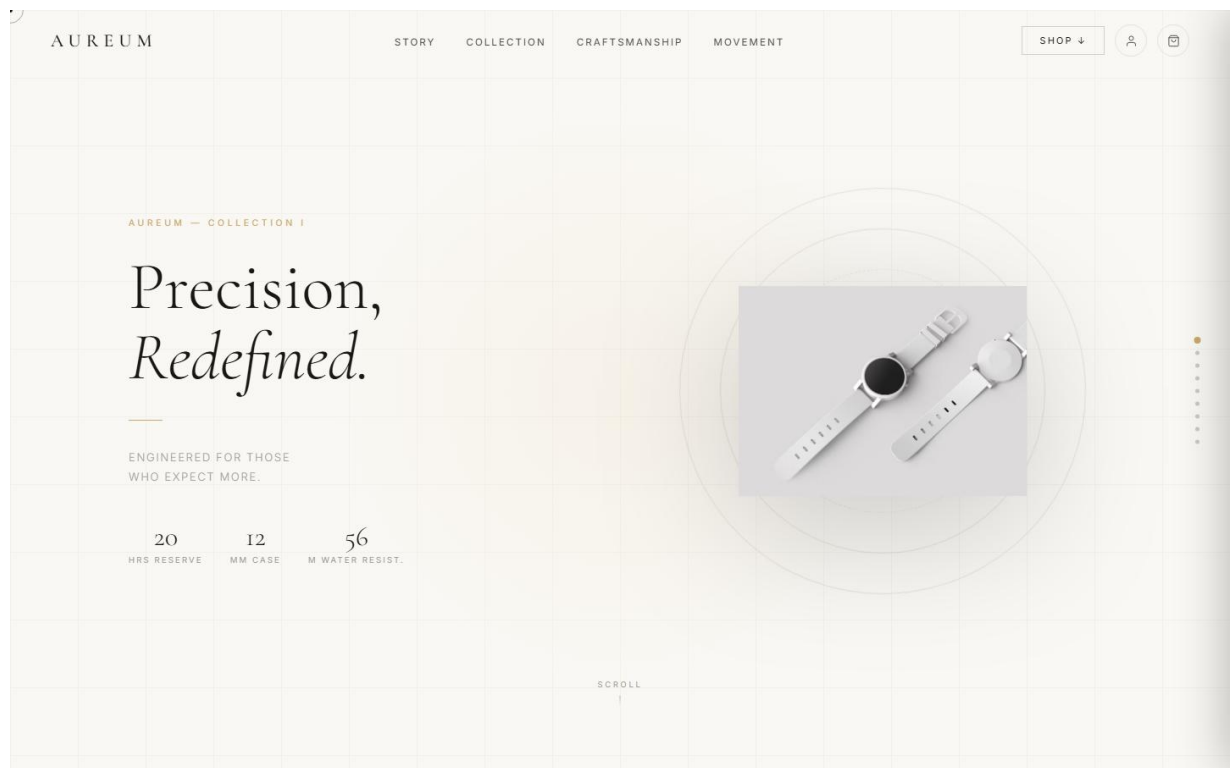


Figure 2: AUREUM Luxury Timepieces static website hero section served via Node.js static middleware on `http://localhost:5000/`, featuring responsive typography, interactive navigation header, animated horology statistics, and product imagery.

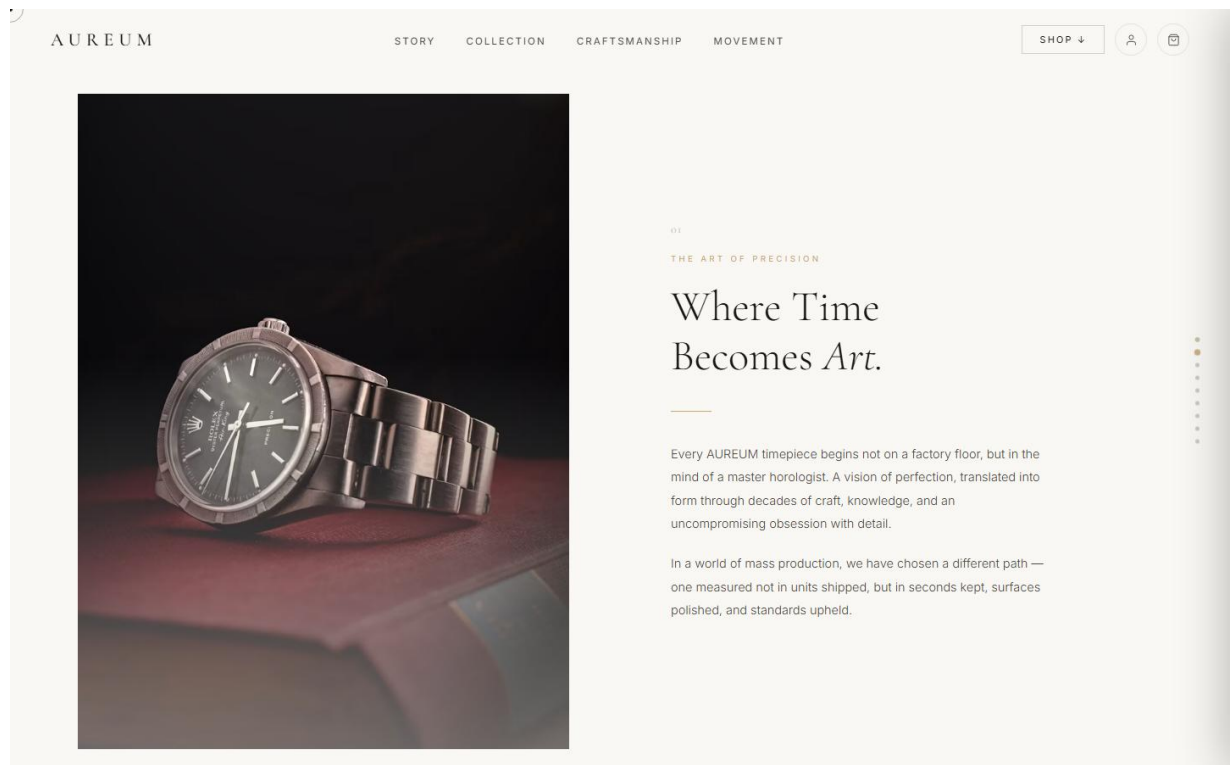


Figure 3: 'The Art of Precision' craftsmanship section served by Node.js, highlighting high-resolution watch photography, horological philosophy, and responsive two-column grid layout.

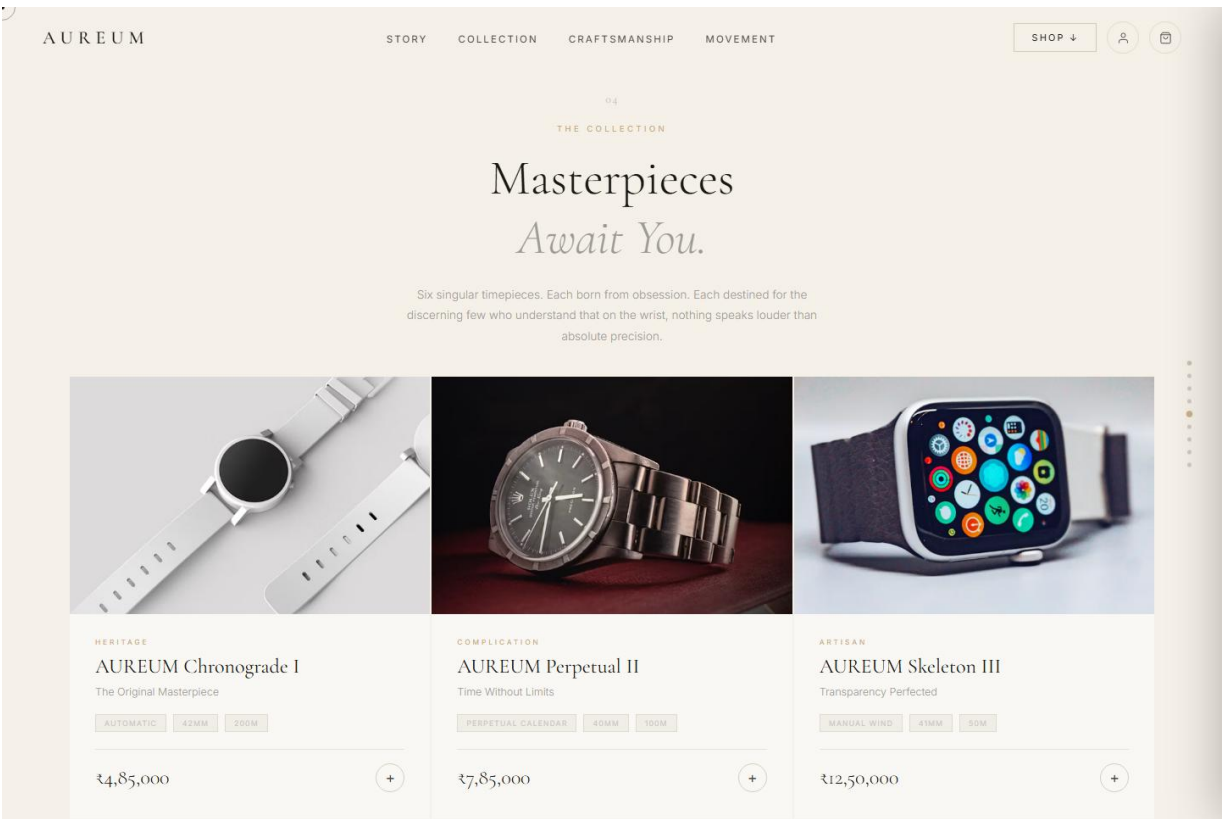


Figure 4: Dynamic luxury timepiece collection catalog served by Node.js, showcasing watch cards (Chronograde I, Perpetual II, Skeleton III) with interactive pricing, technical specifications, and real-time shopping cart integration.

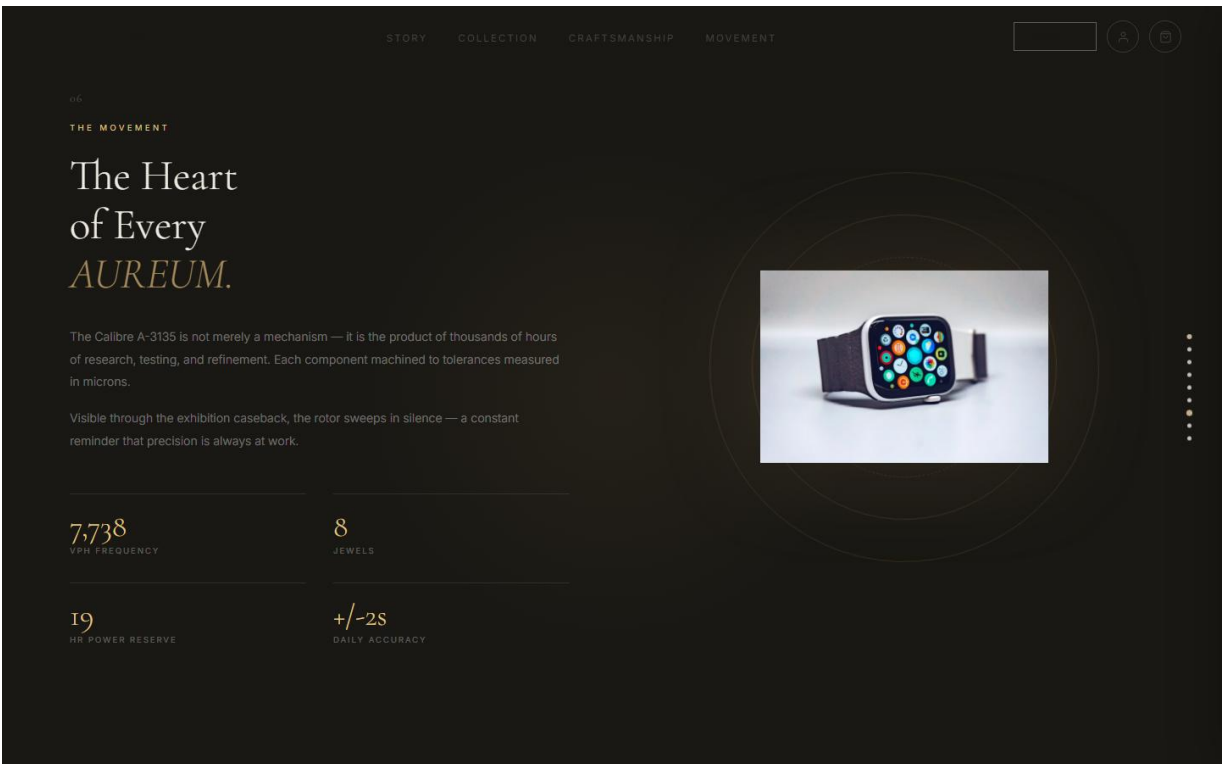


Figure 5: Calibre 8800 mechanical movement showcase section served by Node.js, highlighting technical frequency metrics (28,800 VPH, 42 Jewels, 72-Hour Power Reserve, Co-Axial Escapement).

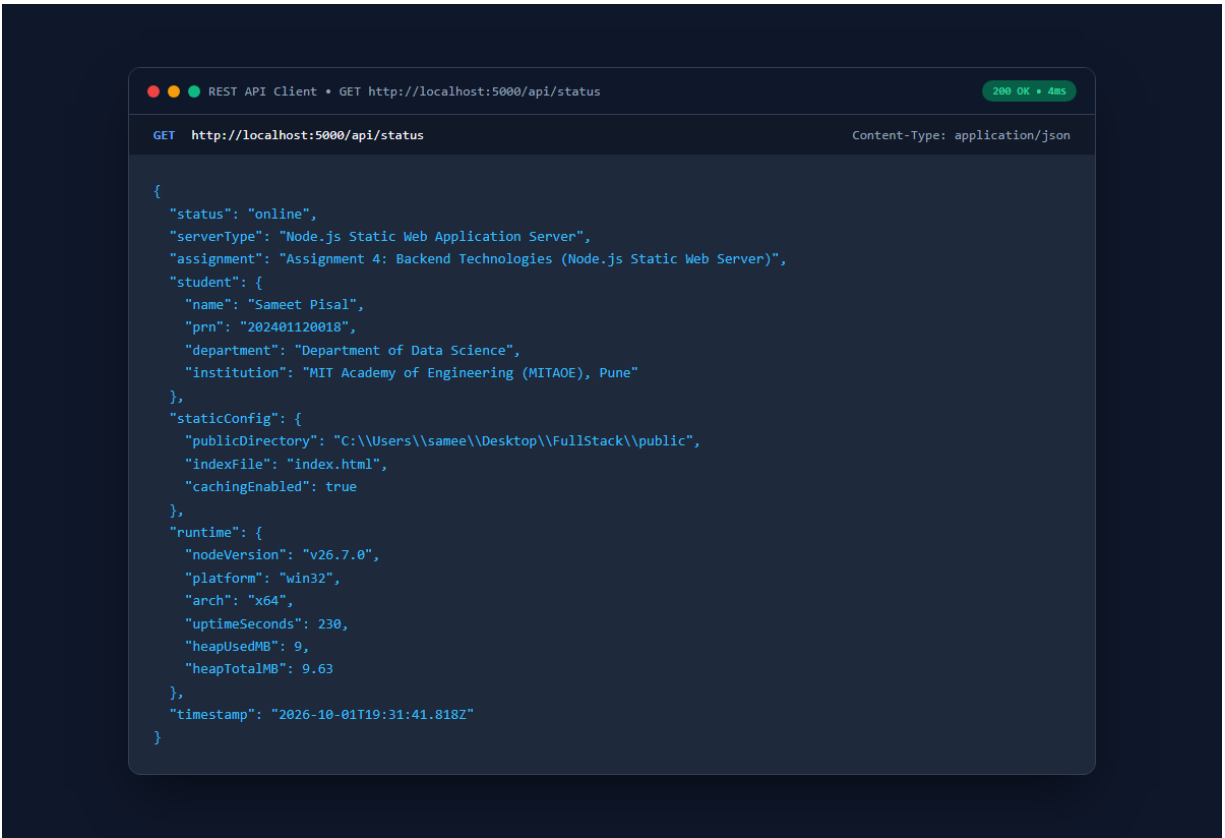


Figure 6: Diagnostic REST API endpoint (GET /api/status) served by Node.js, delivering real-time server health, memory metrics (heap usage), uptime, and student verification credentials (Sameet Pisal, PRN: 202401120018, Department of Data Science).

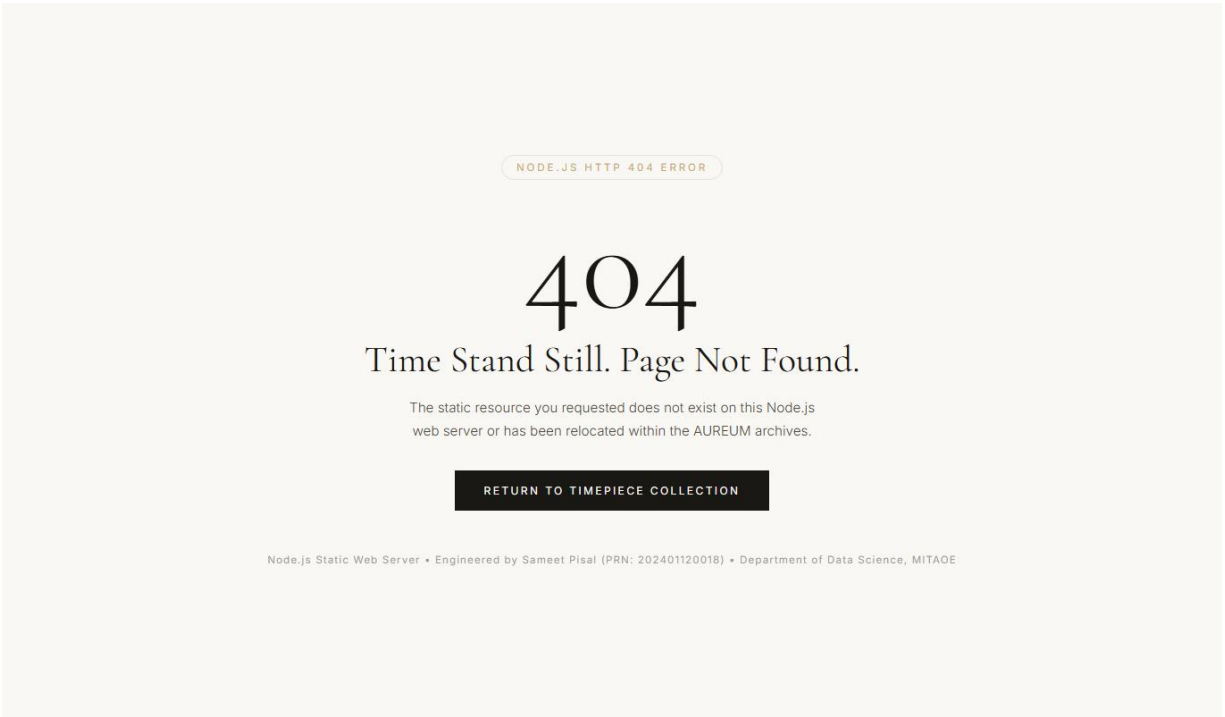


Figure 7: Custom branded HTTP 404 Not Found error page served by Node.js middleware when a non-existent URL route is requested, featuring institutional attribution and return-to-collection navigation.

5. CORE BACKEND CODE IMPLEMENTATION

Key excerpts from server.js demonstrating the production implementation of the Node.js static web server:

Snippet 1: Server Initialization, Environment Configuration & Port Binding

```
/**
 * MIT ACADEMY OF ENGINEERING (MITAOE), PUNE
 * DEPARTMENT OF DATA SCIENCE
 * Course: Full Stack Web Development - Laboratory Assignment 4
 * Student Developer: Sameet Pisal | PRN: 202401120018
 */
const express = require('express');
const path = require('path');
const fs = require('fs');

const app = express();
const PORT = process.env.PORT || 5000;
const PUBLIC_DIR = path.join(__dirname, 'public');
```

Snippet 2: HTTP Request Logging & Real-Time Latency Middleware

```
// Intercepts all requests, tracks start time, and logs formatted latency on finish
app.use((req, res, next) => {
  const start = Date.now();
  const timestamp = new Date().toLocaleTimeString();

  res.on('finish', () => {
    const duration = Date.now() - start;
    const statusCode = res.statusCode;
    const color = statusCode >= 400 ? '\x1b[31m' : statusCode >= 300 ? '\x1b[33m' : '\x1b[32m';
    console.log(`[${timestamp}] ${req.method} ${req.originalUrl} -> ${color}${statusCode}\x1b[0m`);
  });

  next();
});
```

Snippet 3: Security & Department of Data Science Metadata Headers

```
// Inject institutional accreditation and developer signature headers into all responses
app.use((req, res, next) => {
  res.setHeader('X-Powered-By', 'Node.js / Express (MITAOE Data Science)');
  res.setHeader('X-Developer', 'Sameet Pisal (PRN: 202401120018)');
  res.setHeader('X-Course', 'Full Stack Web Development - Assignment 4');
  res.setHeader('X-Department', 'Department of Data Science');
  next();
});
```

Snippet 4: Static File Serving Middleware with HTTP Cache Control

```
// Serves static website files (HTML, CSS, JS, Images) from the public/ directory
app.use(express.static(PUBLIC_DIR, {
```

```

    maxAge: '1h',          // Enable browser caching for 1 hour
    etag: true,            // Generate ETag checksums for cache revalidation (HTTP 304)
    index: 'index.html' // Automatically resolve root requests to index.html
  }));

// Fallback static serving for root directory assets
app.use(express.static(__dirname));

```

Snippet 5: Diagnostic Server Health & Timepiece Catalog REST APIs

```

// Server Health & Institutional Diagnostic Endpoint
app.get('/api/status', (req, res) => {
  const mem = process.memoryUsage();
  res.json({
    status: 'online',
    serverType: 'Node.js Static Web Application Server',
    assignment: 'Assignment 4: Backend Technologies (Node.js Static Web Server)',
    student: {
      name: 'Sameet Pisal',
      prn: '202401120018',
      department: 'Department of Data Science',
      institution: 'MIT Academy of Engineering (MITAOE), Pune'
    },
    staticConfig: {
      publicDirectory: PUBLIC_DIR,
      indexFile: 'index.html',
      cachingEnabled: true
    },
    runtime: {
      nodeVersion: process.version,
      platform: process.platform,
      arch: process.arch,
      uptimeSeconds: Math.floor(process.uptime()),
      heapUsedMB: Math.round((mem.heapUsed / 1024 / 1024) * 100) / 100,
      heapTotalMB: Math.round((mem.heapTotal / 1024 / 1024) * 100) / 100
    },
    timestamp: new Date().toISOString()
  });
});

// Watch Collection REST API Endpoint
app.get('/api/watch-collection', (req, res) => {
  res.json({
    brand: 'AUREUM',
    count: 6,
    watches: [
      { id: 1, name: 'Chronograde I', price: 485000, movement: 'Automatic' },
      { id: 2, name: 'Perpetual II', price: 785000, movement: 'Perpetual Calendar' },
      { id: 3, name: 'Skeleton III', price: 1250000, movement: 'Manual Wind' }
    ]
  });
});

```

Snippet 6: Custom Branded HTTP 404 Not Found Middleware

```
// Gracefully handle unmapped routes by serving the branded 404.html page
app.use((req, res) => {
  const custom404 = path.join(PUBLIC_DIR, '404.html');
  if (fs.existsSync(custom404)) {
    res.status(404).sendFile(custom404);
  } else {
    res.status(404).send('<h1>404 - Page Not Found</h1>');
  }
});

// Server listener and ASCII console banner
app.listen(PORT, () => {
  console.log(`Server successfully started on http://localhost:${PORT}`);
});
```

6. CONCLUSION & LEARNING OUTCOMES

Through the successful completion of Laboratory Assignment 4, the following core engineering competencies in backend web technologies were thoroughly demonstrated and mastered:

1. Asynchronous Node.js Server Architecture: Acquired in-depth practical expertise in leveraging the Node.js event-driven runtime to construct high-throughput HTTP servers capable of serving multi-resource static web applications without blocking the main event thread.
2. Express.js Middleware Mechanics: Mastered the implementation of chained middleware pipelines for request logging, response latency measurement, HTTP header injection, and centralized error handling.
3. Static Asset Optimization & Cache Control: Understood how static web servers utilize MIME type resolution, ETag checksum headers, and HTTP max-age Cache-Control directives to minimize bandwidth consumption and enhance client-side load performance.
4. RESTful Diagnostics & Operational Observability: Implemented structured JSON health endpoints exposing Node.js process runtime statistics (memory usage, heap allocations, uptime, and system platform) for production server observability.
5. Error Resilience & User Experience: Built custom 404 error interceptors that maintain brand continuity and provide intuitive navigation fallbacks when requested resources do not exist on the server.